

CAMUNDA CHAPTER MUNICH

Mastering the distributed transaction problem

What changes when you move from Camunda 7 to Zeebe, and how to keep your data consistent

MIRAGON

Marco Schäck, Miragon GmbH



YOUR SPEAKER

Marco Schäck

Process Development Lead, Miragon

Contact me

[!\[\]\(339a16584d5da0f0a3ca4e9ec17bf6a1_img.jpg\) marco.schaeck@miragon.io](mailto:marco.schaeck@miragon.io)

[!\[\]\(a870788d6ed9b8fd294b7654a8c8526b_img.jpg\) linkedin.com/in/schaeckm](https://www.linkedin.com/in/schaeckm)

MEET THE BRAND

MiraVelo and the Inner Circle

MiraVelo leases lifestyle bikes for the quarter-life crisis. The Inner Circle is its exclusive membership.



The brand

Lifestyle bike leasing

Gravel bikes for the weekend, road bikes for the asphalt, leased instead of bought.



Inner Circle

The membership

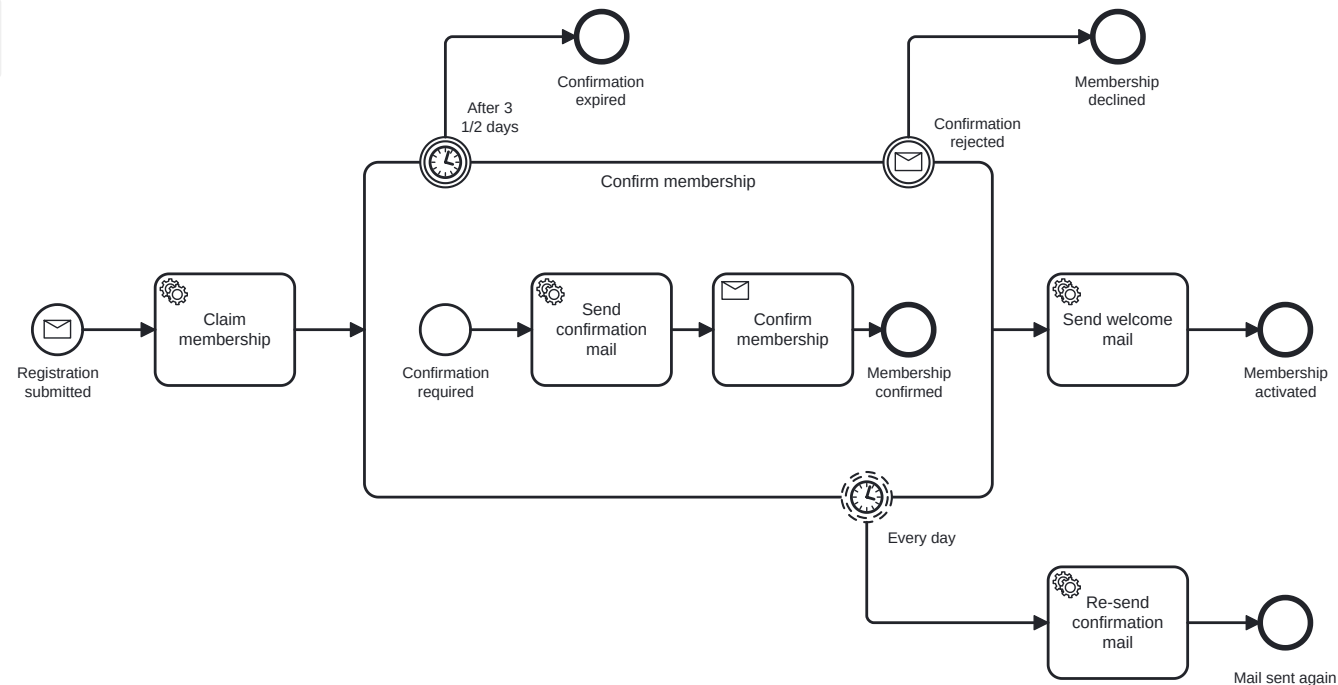
An exclusive club with perks, joined through a double opt-in flow.

OUR RUNNING EXAMPLE

MiraVelo's exclusive Inner Circle

Token Simulation 

 Expand



BPMN.iO

Register, confirm within a few days, get a welcome mail. Until then: one reminder per day.

Welcomed, but not confirmed



The user says



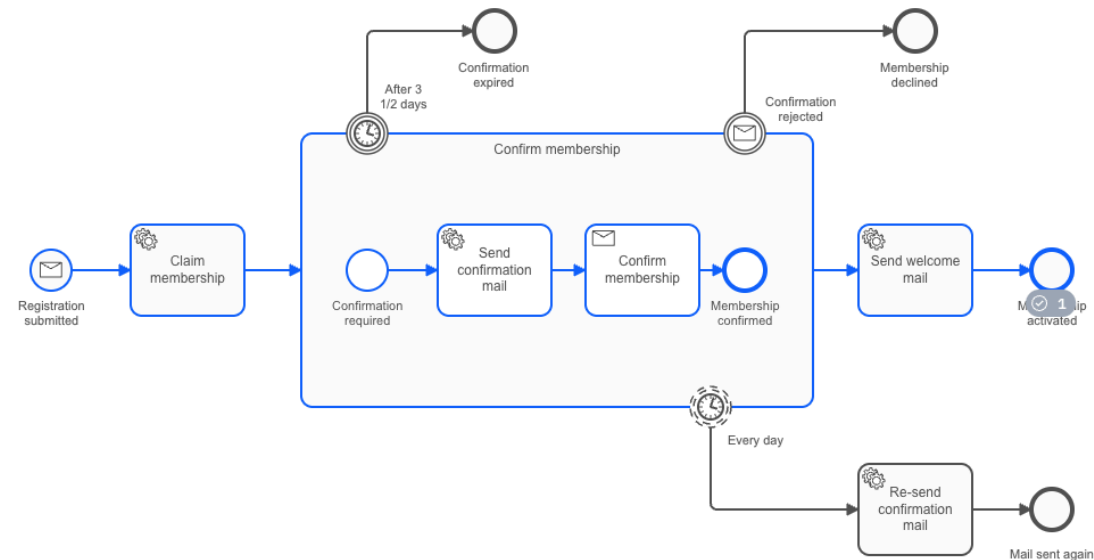
The database says



The engine says

Instance completed

Every step ran through to the end



Registered, but no mail at all



The user says



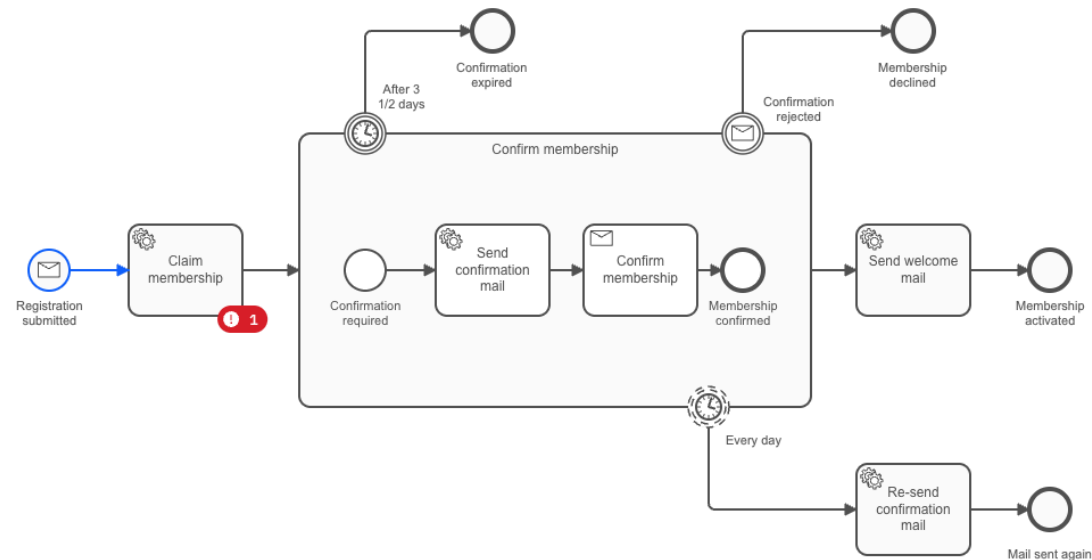
The database says



The engine says

Instance with incident

It is stuck on its very first task



The worker cannot find the membership

Element "Claim membership" Error

```
1 java.util.NoSuchElementException
2 at io.miragon.example.adapter.out.db.MembershipPersistenceAdapter.find(MembershipPersistenceAdapter.kt:14)
3 at io.miragon.example.application.service.ClaimMembershipService.claim(ClaimMembershipService.kt:21)
4 at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
5 at java.base/java.lang.reflect.Method.invoke(Method.java:580)
6 at org.springframework.aop.support.AopUtils.invokeJoinpointUsingReflection(AopUtils.java:359)
7 at org.springframework.aop.framework.ReflectiveMethodInvocation.invokeJoinpoint(ReflectiveMethodInvocation.java:190)
8 at org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:158)
9 at org.springframework.transaction.interceptor.TransactionInterceptor$1.proceedWithInvocation(TransactionInterceptor.java:133)
10 at org.springframework.transaction.interceptor.TransactionAspectSupport.invokeWithinTransaction(TransactionAspectSupport.java:371)
11 at org.springframework.transaction.interceptor.TransactionInterceptor.invoke(TransactionInterceptor.java:130)
12 at org.springframework.aop.framework.ReflectiveMethodInvocation.proceed(ReflectiveMethodInvocation.java:179)
13 at org.springframework.aop.framework.CglibAopProxy$DynamicAdvisedInterceptor.intercept(CglibAopProxy.java:719)
14 at io.miragon.example.application.service.ClaimMembershipService$$SpringCGLIB$$0.claim(<generated>)
15 at io.miragon.example.adapter.in.zeebe.ClaimMembershipWorker.claimMembership(ClaimMembershipWorker.kt:26)
16 at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
17 at java.base/java.lang.reflect.Method.invoke(Method.java:580)
18 at io.camunda.client.spring.bean.SpringMethodInfo.invoke(SpringMethodInfo.java:80)
19 at io.camunda.client.jobhandling.JobHandlerInvokingBeans.lambda$handle$0(JobHandlerInvokingBeans.java:76)
20 at io.micrometer.core.instrument.AbstractTimer.recordCallable(AbstractTimer.java:192)
21 at io.camunda.client.metrics.MicrometerMetricsRecorder.executeWithTimer(MicrometerMetricsRecorder.java:70)
22 at io.camunda.client.jobhandling.JobHandlerInvokingBeans.handle(JobHandlerInvokingBeans.java:75)
23 at io.camunda.client.impl.worker.JobRunnableFactoryImpl.executeJob(JobRunnableFactoryImpl.java:61)
24 at io.camunda.client.impl.worker.JobRunnableFactoryImpl.lambda$create$0(JobRunnableFactoryImpl.java:49)
25 at io.micrometer.core.instrument.internal.TimedRunnable.run(TimedRunnable.java:49)
26 at java.base/java.util.concurrent.Executors$RunnableAdapter.call(Executors.java:572)
27 at java.base/java.util.concurrent.FutureTask.run(FutureTask.java:317)
28 at java.base/java.util.concurrent.ScheduledThreadPoolExecutor$ScheduledFutureTask.run(ScheduledThreadPoolExecutor.java:304)
29 at java.base/java.util.concurrent.ThreadPoolExecutor.runWorker(ThreadPoolExecutor.java:1144)
30 at java.base/java.util.concurrent.ThreadPoolExecutor$Worker.run(ThreadPoolExecutor.java:642)
```

NoSuchElementException: the membership is not in the database.



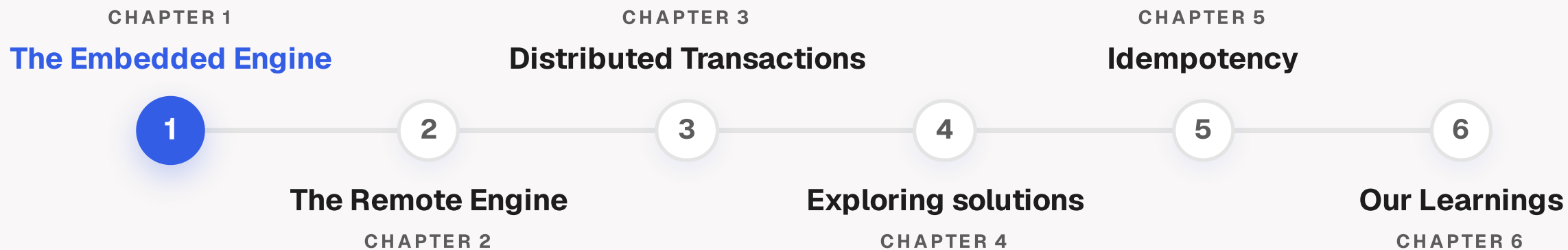
TWO TICKETS, ONE MIGRATION

So what has **actually** **changed?**

To answer that, we first look back at Camunda 7, and then at what Zeebe does differently

CHAPTER OVERVIEW

What this talk will cover

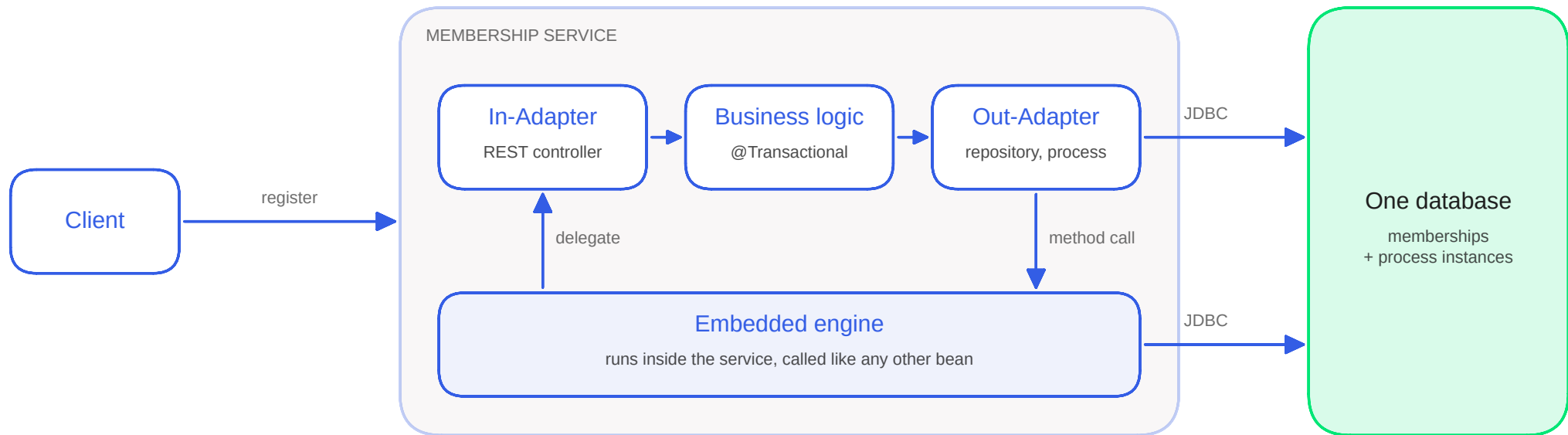


CHAPTER 1

The Embedded Engine

How we knew it

The engine lives inside your service

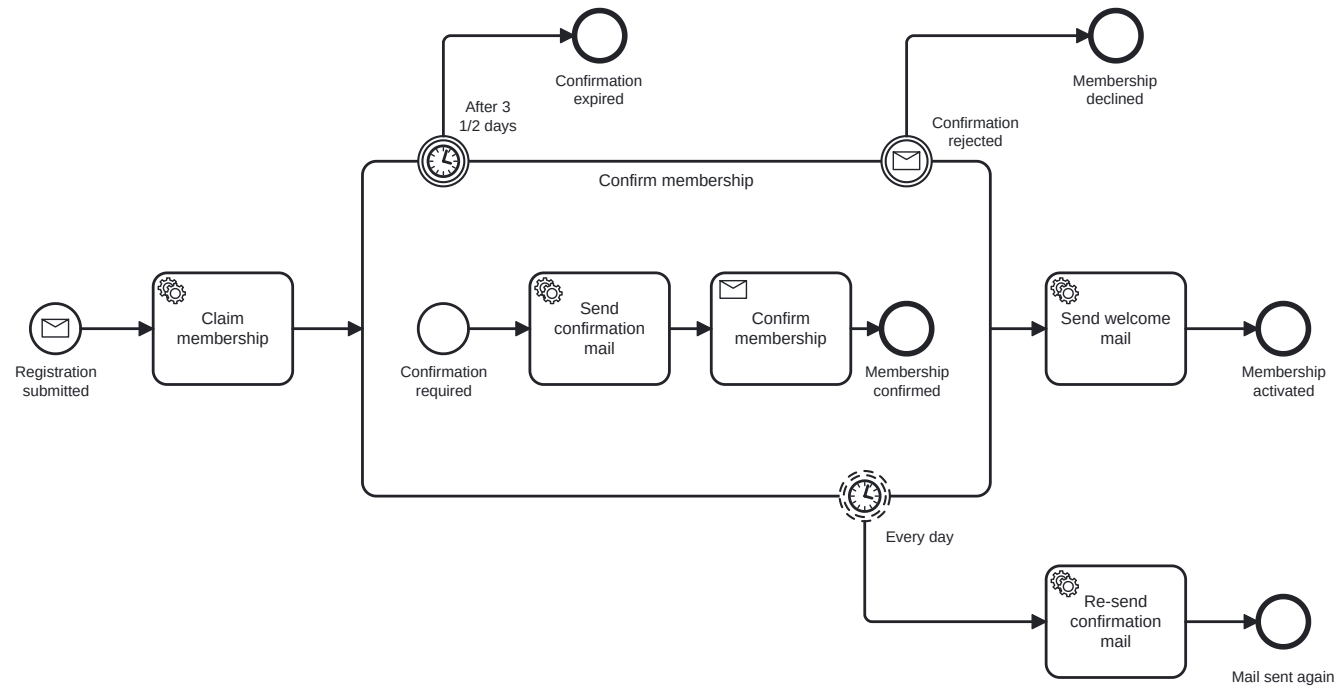


No network in between, and one database for both

The process ships with the service

Token Simulation 

 Expand



BPMN.iO

Modelled once, deployed by the engine when the service starts

Automating it took little code

Three kinds of classes. Delegates run the tasks, adapters talk to the engine, services hold the logic

SendWelcomeMailDelegate.kt

MembershipProcessAdapter.kt

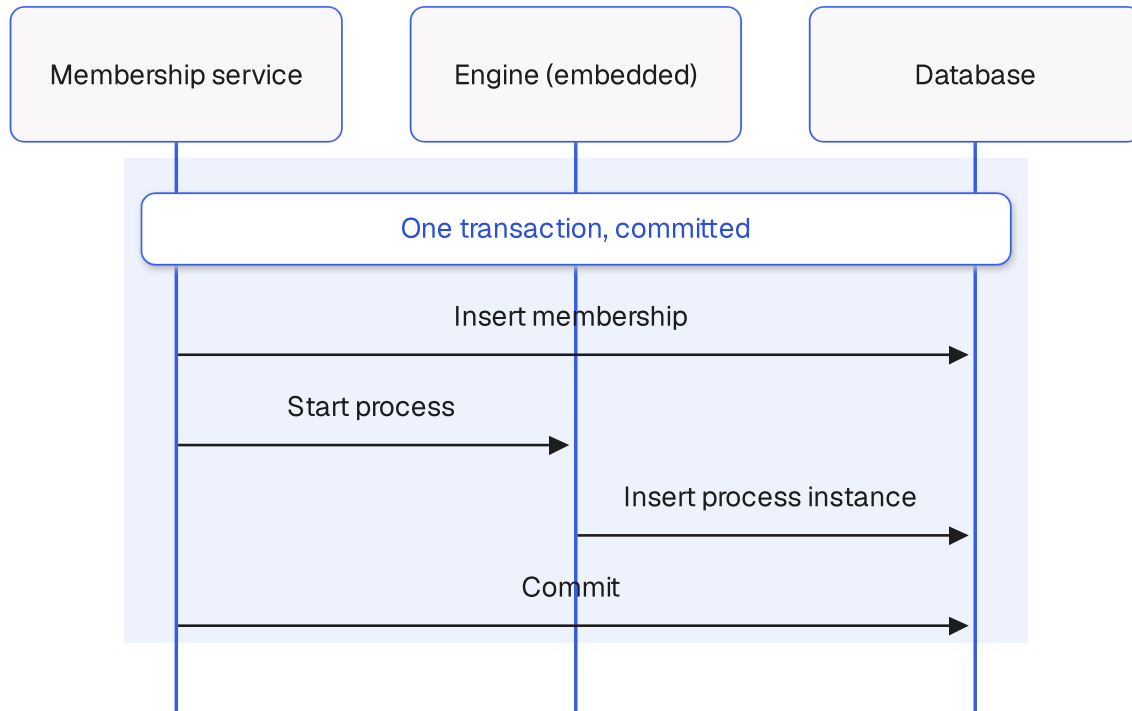
RegisterMembershipService.kt

KOTLIN

```
@Service
@Transactional
class RegisterMembershipService(
    private val repository: MembershipRepository,
    private val processPort: MembershipProcess
) : RegisterMembershipUseCase {

    override fun register(command: RegisterMembershipUseCase.Command): MembershipId {
        val membership = buildMembership(command)
        repository.save(membership)
        processPort.submitRegistration(membership.id)
        return membership.id
    }
}
```

When it works, everyone agrees



The user says

I am a member

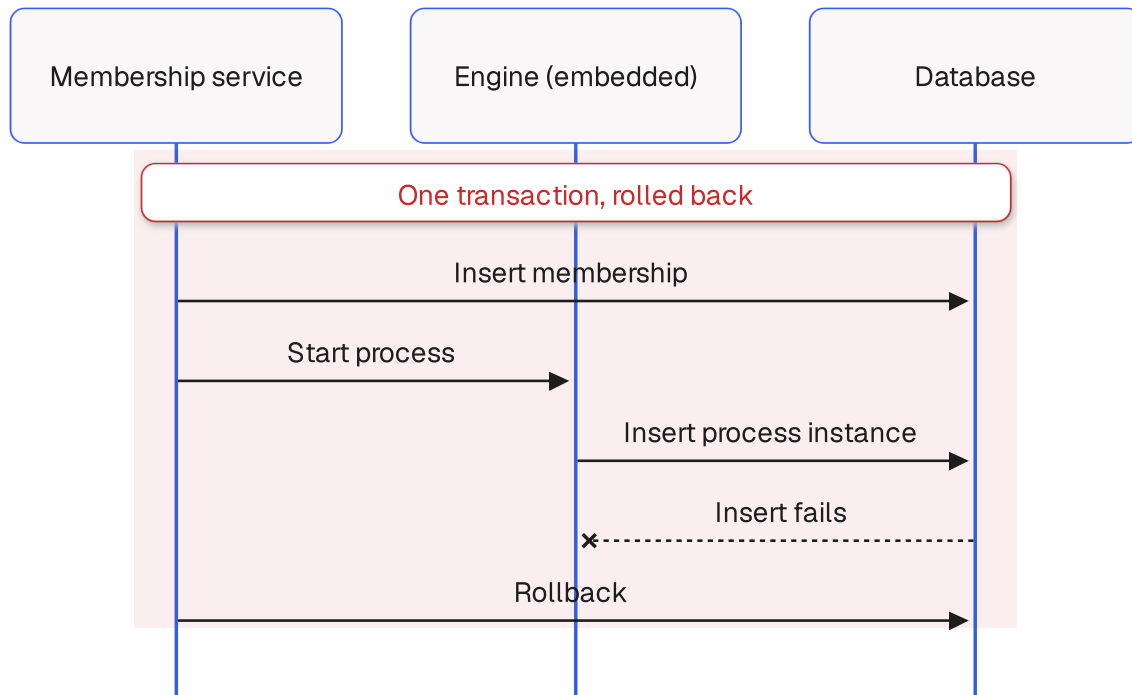
The database says

Membership stored

The engine says

Process instance running

When it fails, everyone still agrees



The user says

It did not work, I will try again

The database says

No membership

The engine says

No process instance



OUR SAFEGUARD

All or nothing

— We never wrote that rule. The database enforces it, all we added was `@Transactional`

Our safeguard: ACID



Atomicity



Consistency



Isolation



Durability

- **This means:** Once committed, the data stays stored
- **Even if** the database crashes right afterwards

CHAPTER 2

The Remote Engine

Entering a new world

02

Zeebe, a remote engine

A standalone service that can be operated in the cloud (SaaS) or on-premise. **Why migrating to Zeebe?**



Performance

Higher throughput



Scalability

Horizontally scalable



Cloud-native

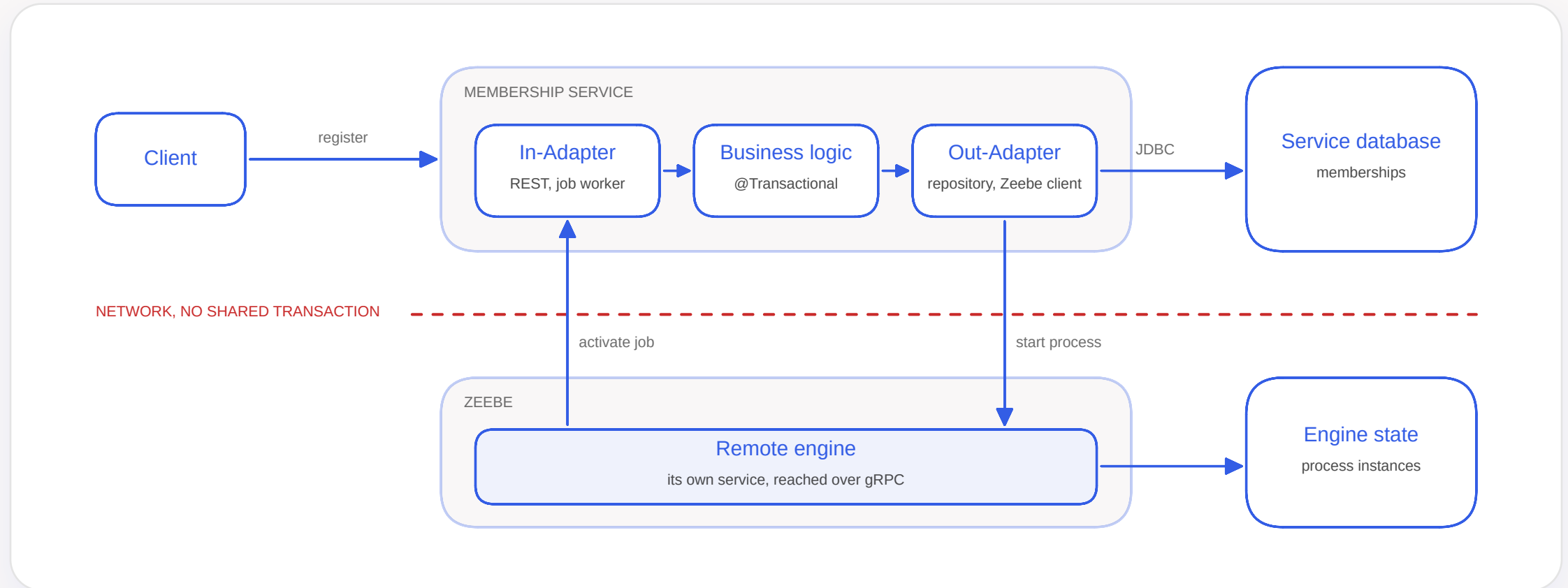
Designed for modern cloud infrastructure (SaaS & On-Prem)



Future-Ready

Modern architecture for future requirements

The engine moves out of your service



Two services, two databases, and a network in between

For developers, it still feels the same

That is why the change is easy to underestimate. The concrete automation follows the same paradigms



Model

Modeling the process with bpmn



Operate

Same tools for operations & modelling



Build

Writing glue-code, testing processes & deploying them

Only the adapters change

The delegate becomes a worker, the adapter talks to Zeebe. Everything else stays as it was

SendWelcomeMailWorker.kt

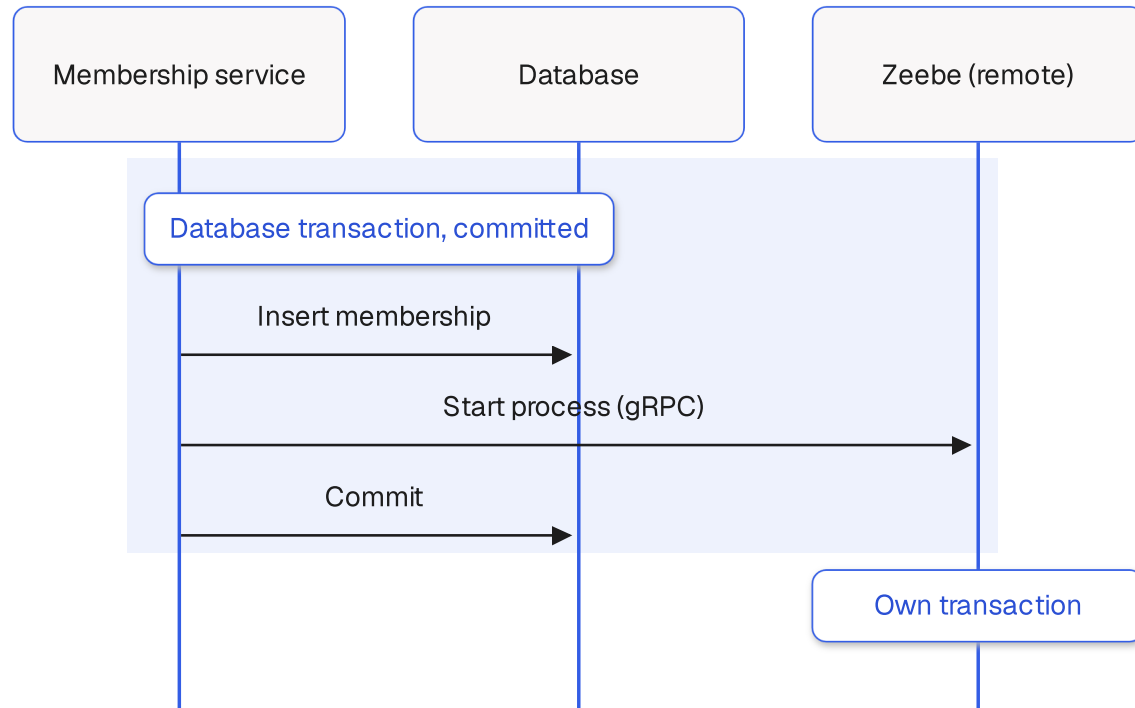
MembershipProcessAdapter.kt

KOTLIN

```
@Component
class MembershipProcessAdapter(
    private val camundaClient: CamundaClient
) : MembershipProcess {

    override fun submitRegistration(id: MembershipId) {
        val variables = mapOf("membershipId" to id.value.toString())
        camundaClient.newPublishMessageCommand()
            .messageName("miravelo.registrationSubmitted")
            .withoutCorrelationKey()
            .variables(variables)
            .send()
            .join()
    }
}
```

When it works, everyone agrees



The user says

I am a member

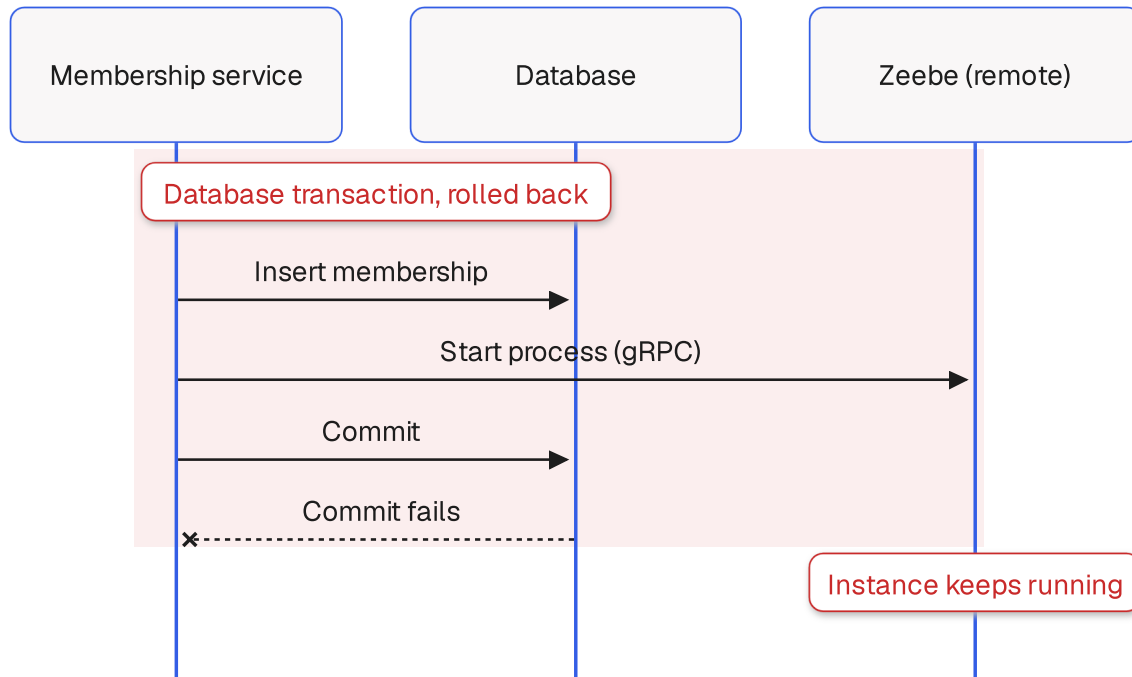
The database says

Membership stored

The engine says

Process instance running

When it fails, nobody agrees



The user says

It did not work, I will try again

The database says

No membership

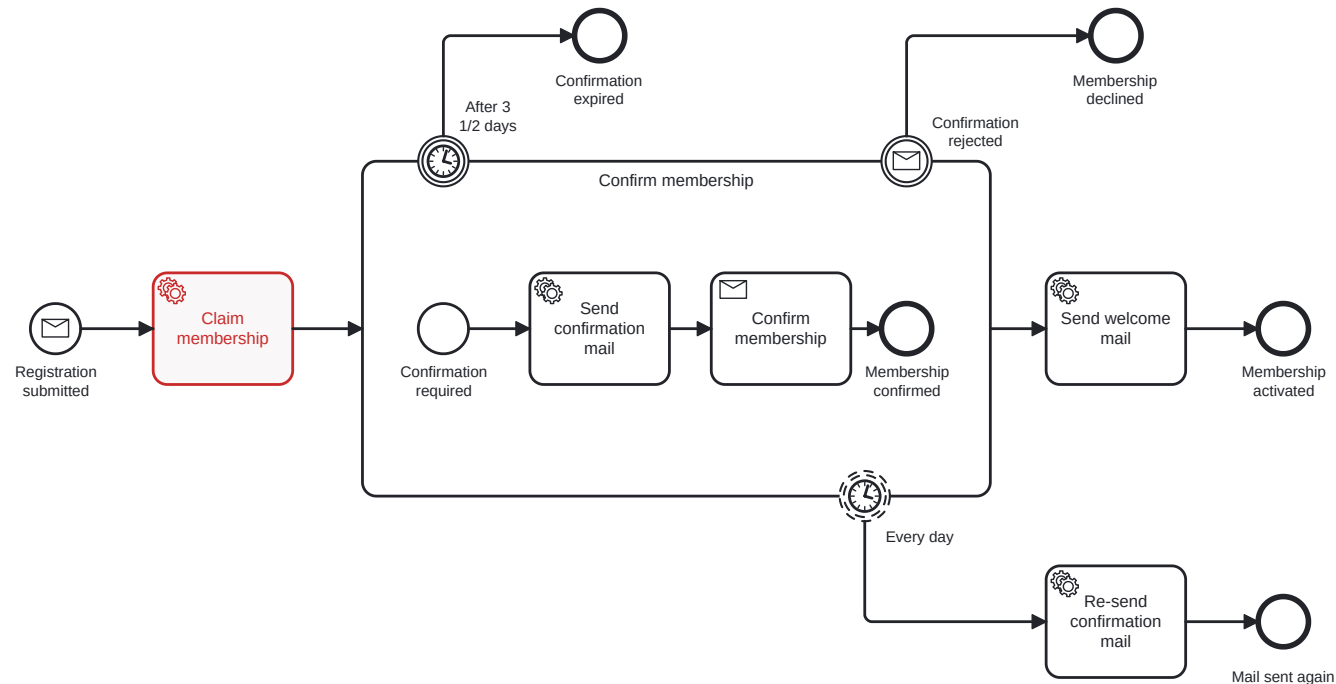
The engine says

Process instance running

And the process runs into an incident

Token Simulation 

 Expand



BPMN.iO

The worker looks for a membership that was rolled back. This is the ticket from the beginning



OUR SAFEGUARD IS GONE

Neither all, **nor nothing**

— `@Transactional` still works, but it only covers our own database now

New paradigm: BASE



Basically Available



Soft State



Eventually Consistent

- **This means:** The systems are not always in sync
- **For a while** different parts may return different data
- **In the end** they converge to the same state

Same failure, different ending

The commit fails in both worlds. Only one of them cleans up after itself



Camunda 7

The database says: no membership

The engine says: no process instance

Result: consistent, the user simply tries again



Zeebe

The database says: no membership

The engine says: process instance running

Result: inconsistent, an incident is waiting

CHAPTER 3

The distributed transaction problem

What we just saw has a name

A generic problem, precisely defined



What



When



Effect

An inconsistent state

If one part of the operation fails, the others do not follow

Four outcomes, two of them hurt

Two systems, each can succeed or fail on its own. That leaves four combinations



A succeeds, B succeeds

Consistent. The happy path



A fails, B fails

Consistent. Nothing happened, try again



A succeeds, B fails

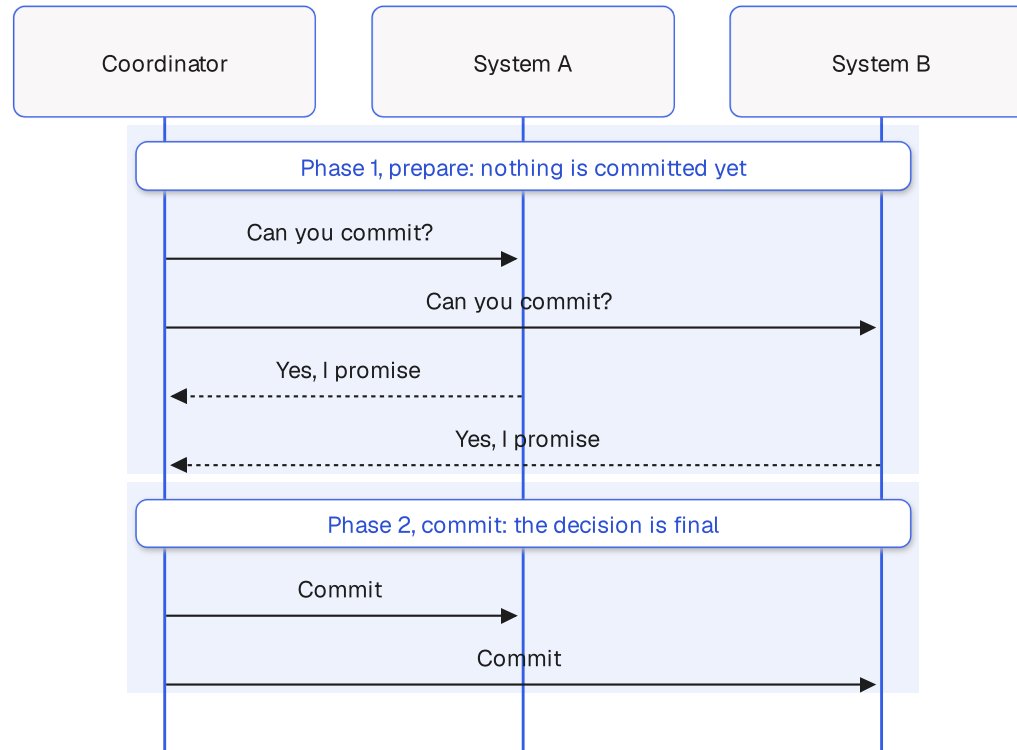
Inconsistent. Later: the service is ahead



A fails, B succeeds

Inconsistent. Later: the engine is ahead

The textbook fix: two-phase commit



Phase 1: ask

Everyone prepares, keeps its locks and votes

Phase 2: tell

All voted yes, so the decision is final

And if a commit fails now?

A yes is a durable promise. Whoever crashes still has to commit after recovery

Why it is not our way out

The guarantee is real, but it is bought with availability



Everyone must speak it

Zeebe, REST APIs and mail servers do not support the protocol



Everyone must wait

Locks are held until the last one has committed



Everyone must be up

One crash blocks all the others

Two-phase commit cannot solve the problem between Zeebe and our database.

So we have to look closely at where exactly it hits us



FROM THEORY TO PRACTICE

Where does it bite in Zeebe?

— Every command we send to the engine crosses a system boundary

Two triggers, three outcomes

Two kinds of commands can leave the system inconsistent



Completing a job

The worker did its work, now it has to tell the engine



Publishing a message

The service tells the engine, then it has to commit its own data

And they leave it in one of three states



Service is ahead of engine

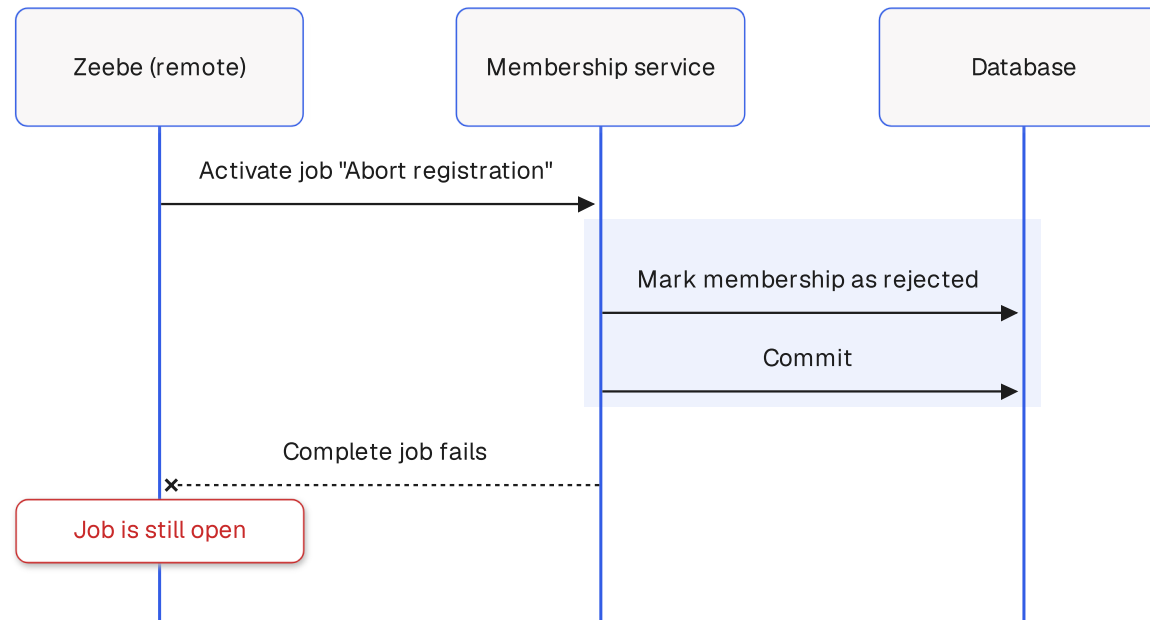


Engine is ahead of service



Engine & service are incompatible

Our data is saved, but the job stays open



The database says

Membership rejected

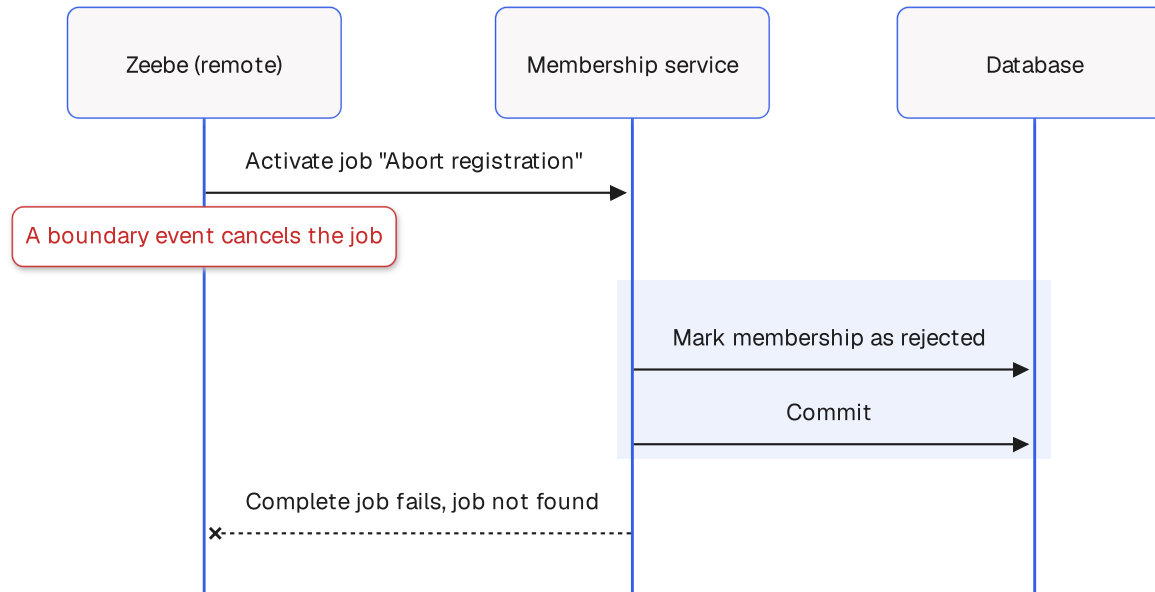
The engine says

Job is still open

Service ahead

Often temporary. A retry fixes it, but may cause side effects

Sometimes no retry can fix it



The database says

Membership rejected

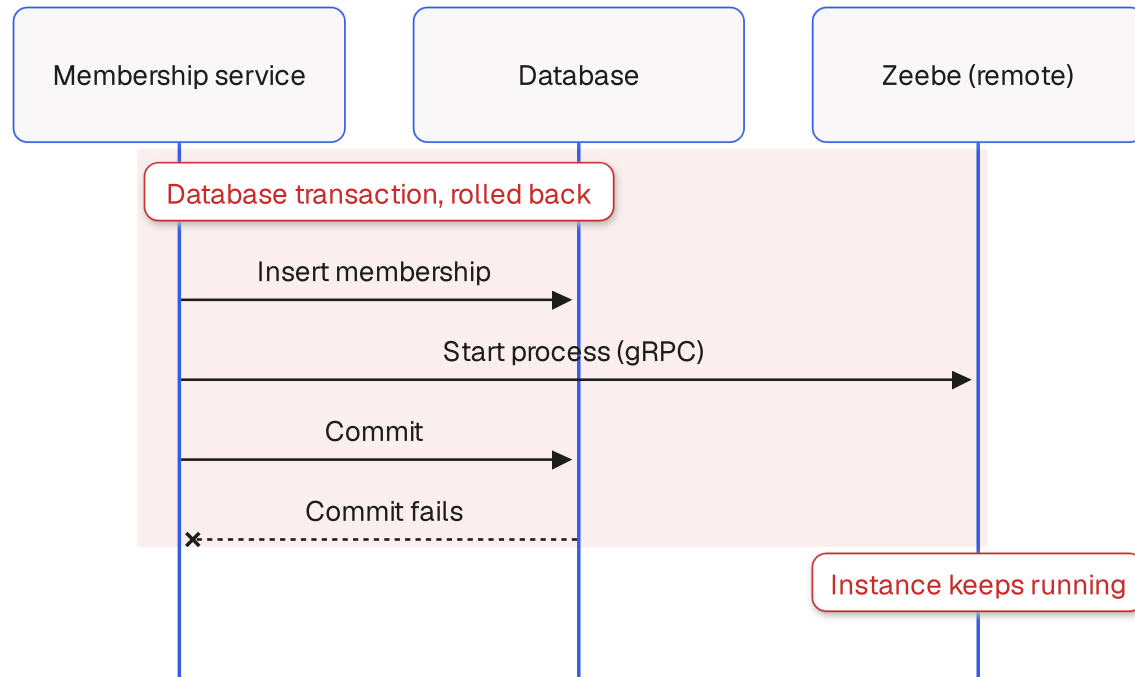
The engine says

Job cancelled by a boundary event

Incompatible

Permanent. No retry will ever succeed

The engine gets the message, our data is lost



The database says

No membership

The engine says

Process instance running

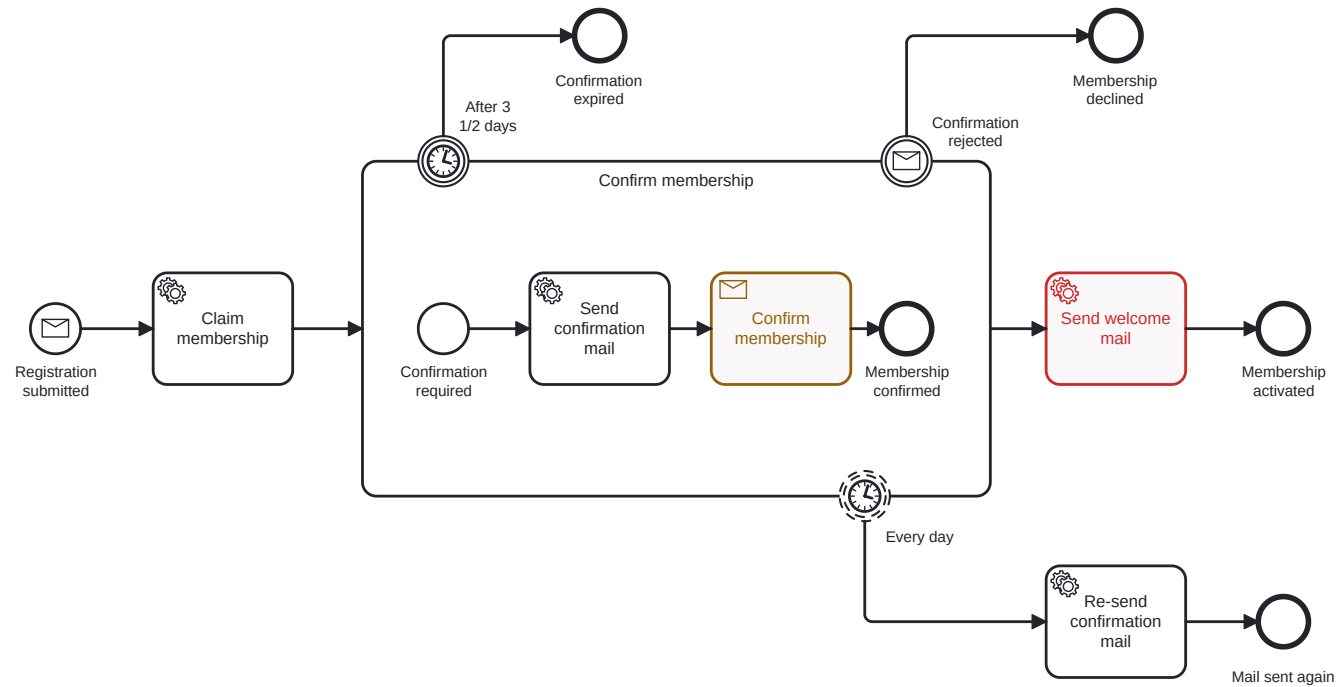
Engine ahead

Workers fail, because the data they need was never saved

It gets worse in the middle of a process

Token Simulation 

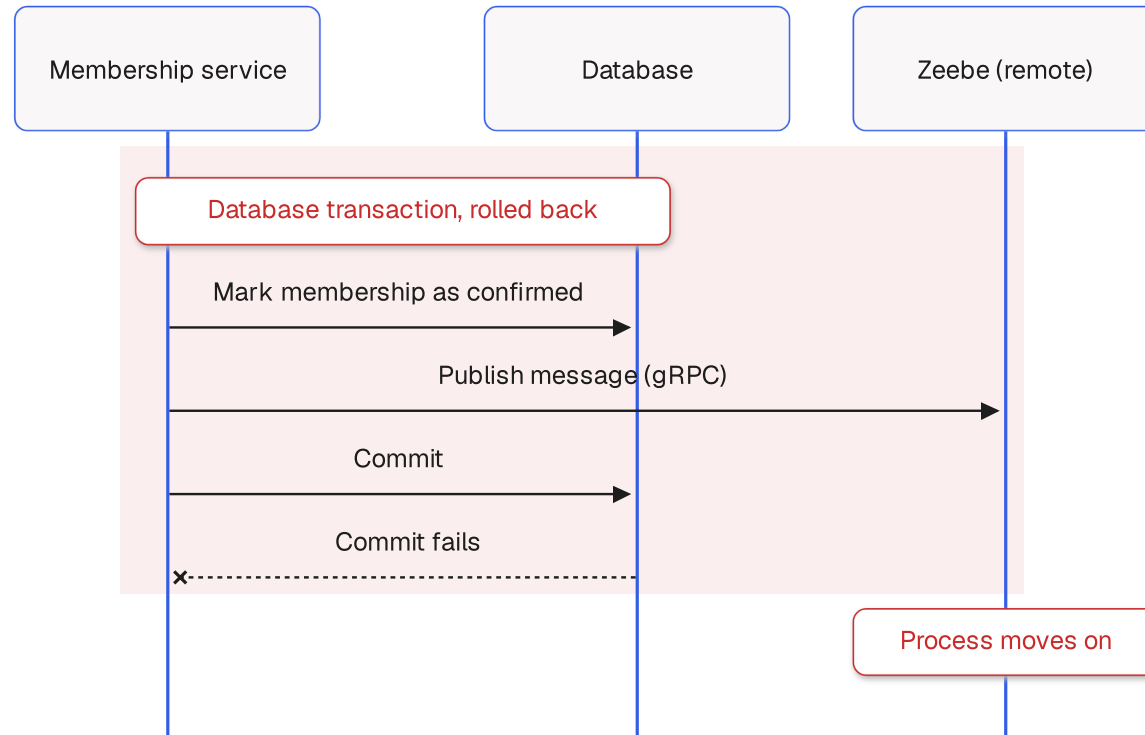
 Expand



BPMN.iO

Take the moment the user confirms the membership

The engine moves on with data we never saved



The database says

Membership not confirmed

The engine says

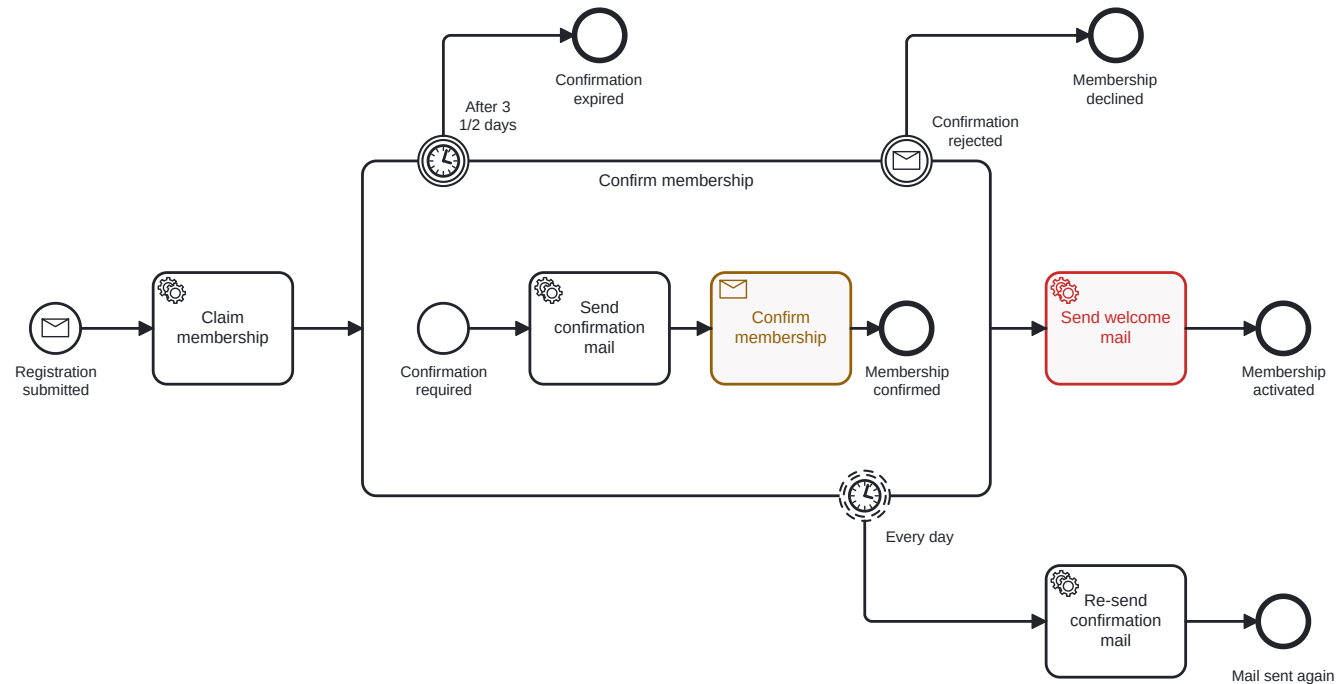
Correlated, next task started

Engine ahead, again

This time the next task finds its data, just not in the right state

And the next task may even succeed

Token Simulation 



BPMN.iO

Incompatible: the welcome mail goes out for a membership that was never confirmed. In other processes such a task could corrupt data

WHAT WE LEARN FROM THIS

It comes down to **timing** and **delivery**

The engine hears of our data before it is saved, or never hears that a job is done

CHAPTER 4

Exploring solutions

Three attempts to narrow the gap



What a solution has to deliver

We judge every attempt by the same three questions. They come straight out of chapter 3



Timing

Does the engine only hear of data that is safely committed?



Delivery

Is the message guaranteed to arrive, even after a crash?



Price

What do we pay for it in code, infrastructure and new problems?

CHAPTER 4 · ATTEMPT 1 OF 3

Retry

If some part fails, just run the whole operation again

4.1

Why a retry feels right

It is the standard answer to failures in one system. It rests on three quiet assumptions



The error is temporary

A network blip or a busy server.
Wait a moment and it works



Repeating is harmless

Doing the same thing twice
leaves the same result as doing it
once

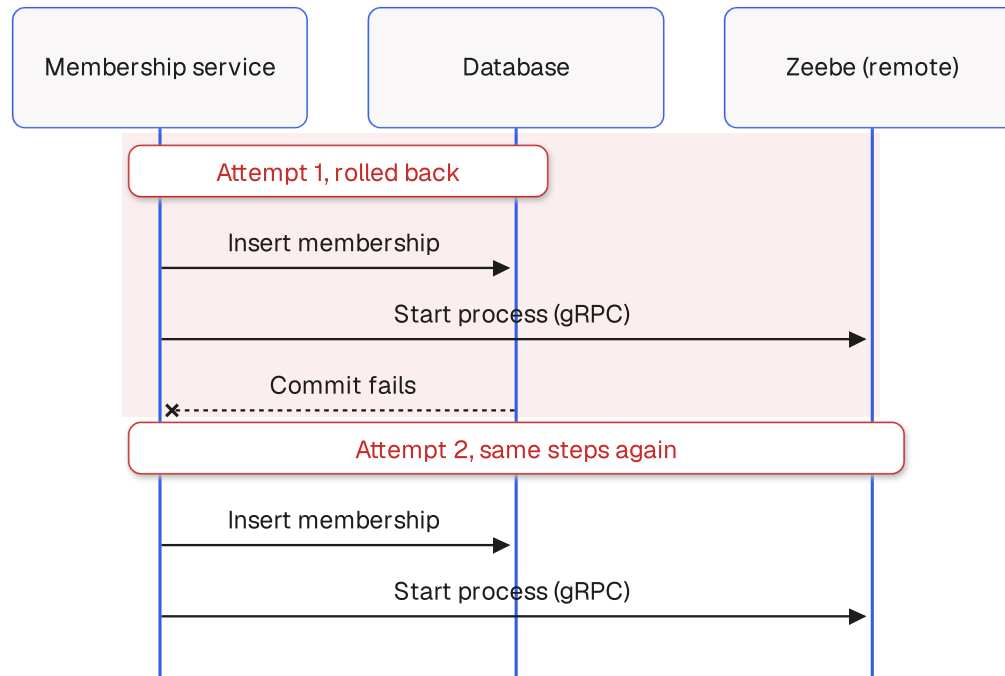


Nothing happened yet

A failed attempt left no traces
anywhere

Across two systems, none of the three is guaranteed

A retry repeats the engine call too



Errors are often permanent

A constraint violation fails on every retry

Retries may start a duplicate

Zeebe does not know the first call belonged to a failed attempt

The order is still wrong

The engine still hears of uncommitted data

Verdict: more problems than it solves

The first thing that comes to mind, and rarely a help. What we take from it: tell Zeebe only after a successful commit



Timing

Unsolved. The engine still runs ahead of our data



Delivery

No guarantee, but duplicates on top



Price

Cheap to build, expensive to operate

CHAPTER 4 · ATTEMPT 2 OF 3

After transaction

Call the engine only once our own data is safe

4.2

Why the order matters

Retry failed because the engine heard of data we never saved. So we turn the order of the two steps around



Commit first

Our own data is safe before anyone else hears of it



Engine second

The call is no longer part of the transaction. It follows a successful commit

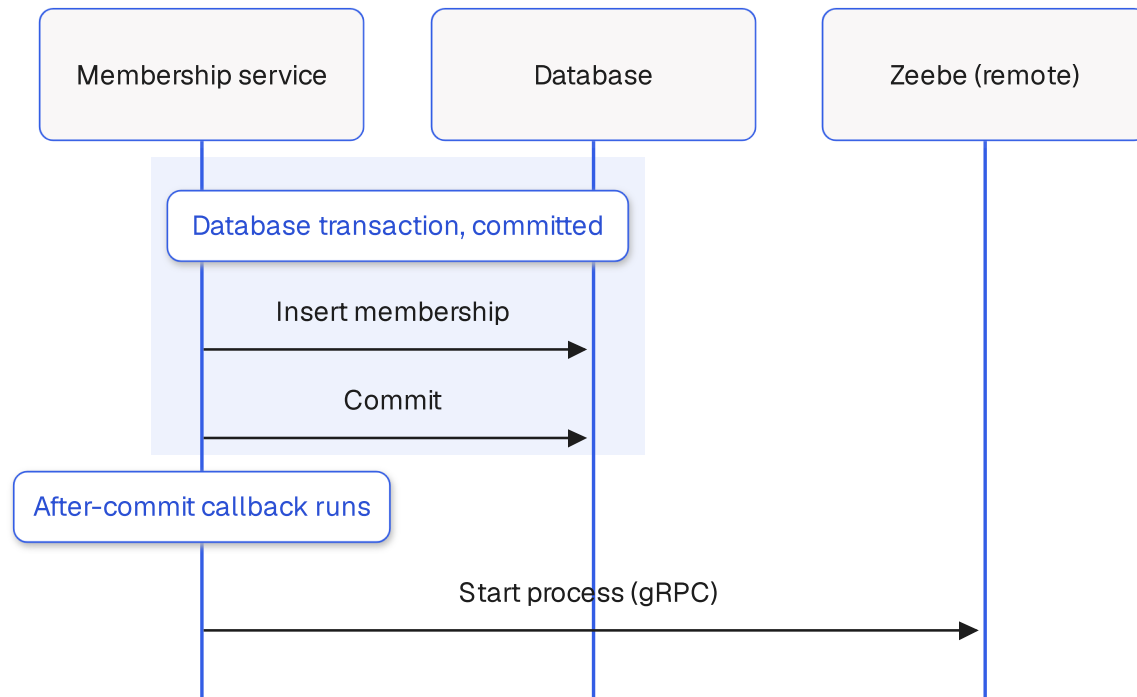


The hook exists

Spring can run code right after a commit. We only have to register it

A failed commit now simply means: nothing happened

The engine only hears of committed data



The database says

Membership stored

The engine says

Process instance running

Timing solved

Zeebe is told after the commit,
never before

The engine call becomes a callback

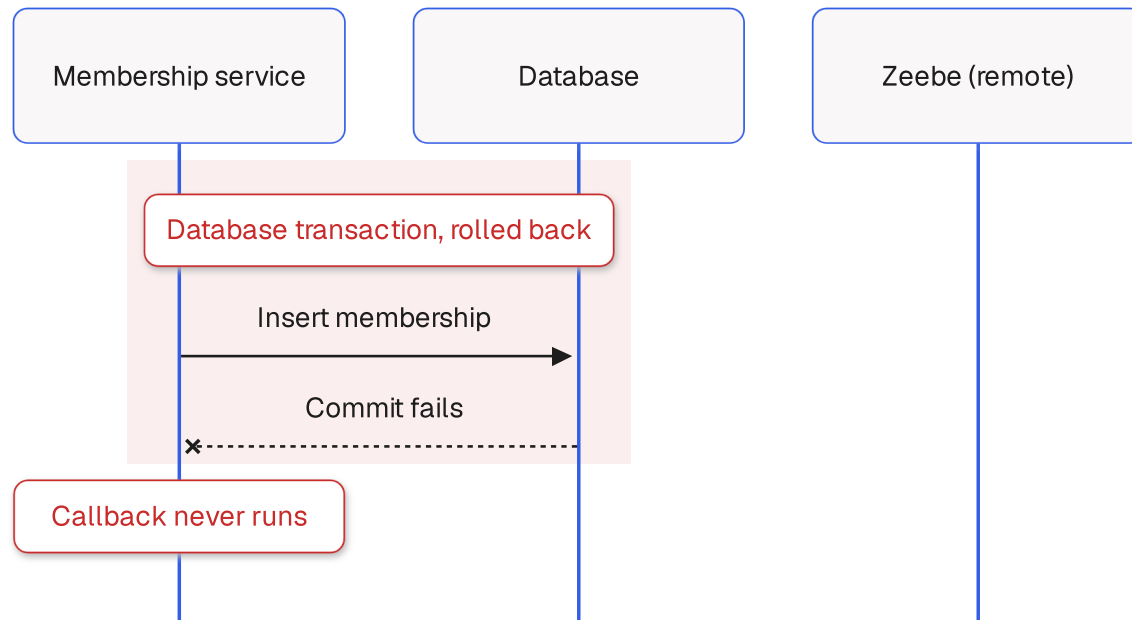
ProcessTransactionManager.kt

KOTLIN

```
fun executeAfterCommit(processEngineCall: () -> Unit) {
    log.debug { "Registering process engine call after commit" }
    registerEngineCallAfterCommit(processEngineCall)
}

private fun registerEngineCallAfterCommit(
    processEngineCall: () -> Unit
) = TransactionSynchronizationManager.registerSynchronization(
    object : TransactionSynchronization {
        override fun afterCommit() = try {
            processEngineCall()
        } catch (e: Exception) {
            val message = "Manual action required. " +
                "Failed to execute engine call"
            log.error(e) { message }
            throw e
        }
    }
)
```

A failing commit never reaches Zeebe



The database says

No membership

The engine says

Nothing, and that is correct

Consistent

The user simply tries again, as in the Camunda 7 days

A health check lowers the risk

BrokerHealthSynchronization.kt

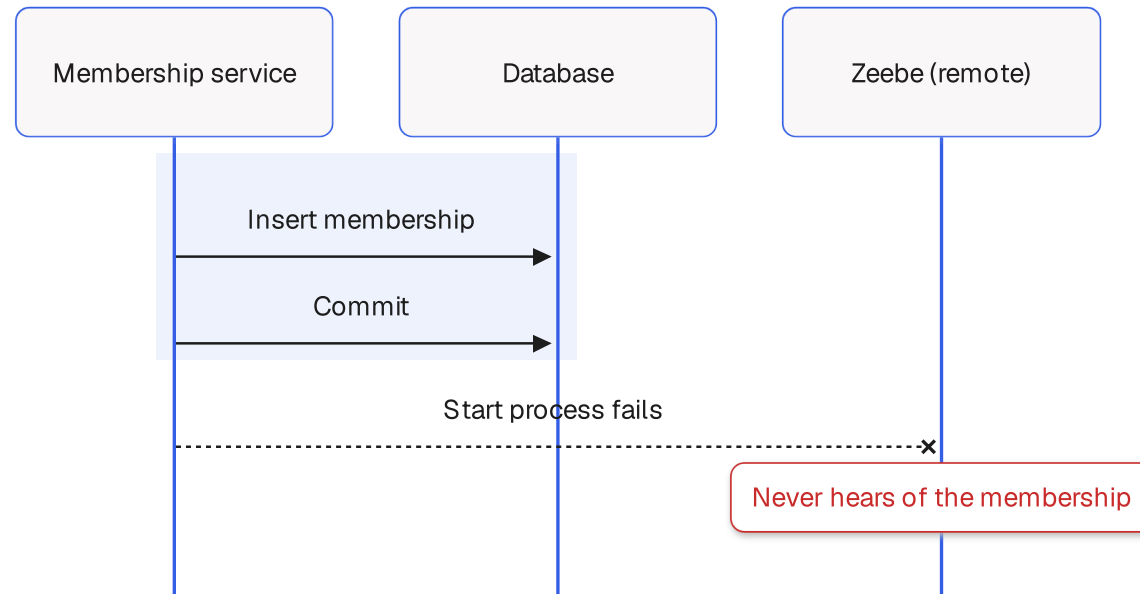
KOTLIN

```
override fun beforeCommit(readOnly: Boolean) {
    val topology = zeebeClient.newTopologyRequest()
        .send()
        .join()
    val healthy = checkBrokerHealth(topology)
    if (!healthy) {
        throw IllegalStateException("No healthy broker found")
    }
}

private fun checkBrokerHealth(topology: Topology): Boolean {
    return topology.brokers.any { brokerIsHealthy(it) }
}

private fun brokerIsHealthy(broker: BrokerInfo): Boolean {
    return broker.partitions.any {
        it.health == PartitionBrokerHealth.HEALTHY
    }
}
```

But the call itself can still fail



The database says

Membership stored

The engine says

Nothing, no process instance

Service ahead

No retry in the pattern. The message is most likely lost

Verdict: easy, but no guarantee

Fine for a proof of concept and uncritical flows. Not for anything that must not get lost



Timing

Solved. The engine never runs ahead of our data



Delivery

Not guaranteed. A failed call or a crash loses the message



Price

Low. One wrapper, optionally a broker health check

CHAPTER 4 · ATTEMPT 3 OF 3

Outbox

Store the message next to the data, deliver it later

4.3

Why a table solves it

After transaction lost the message because it only lived in memory. So we write it down



Stored together

The message goes into our database, in the same transaction as the business data



Survives a crash

What is committed is still there after a restart. Nothing depends on memory any more



Delivered later

A separate processor sends it to Zeebe and keeps trying until it worked

Known as the transactional outbox pattern

Two systems become one again

The message goes into our own database, in the same transaction as the business data. That gives us back the ACID of Camunda 7



Phase 1: initial transaction

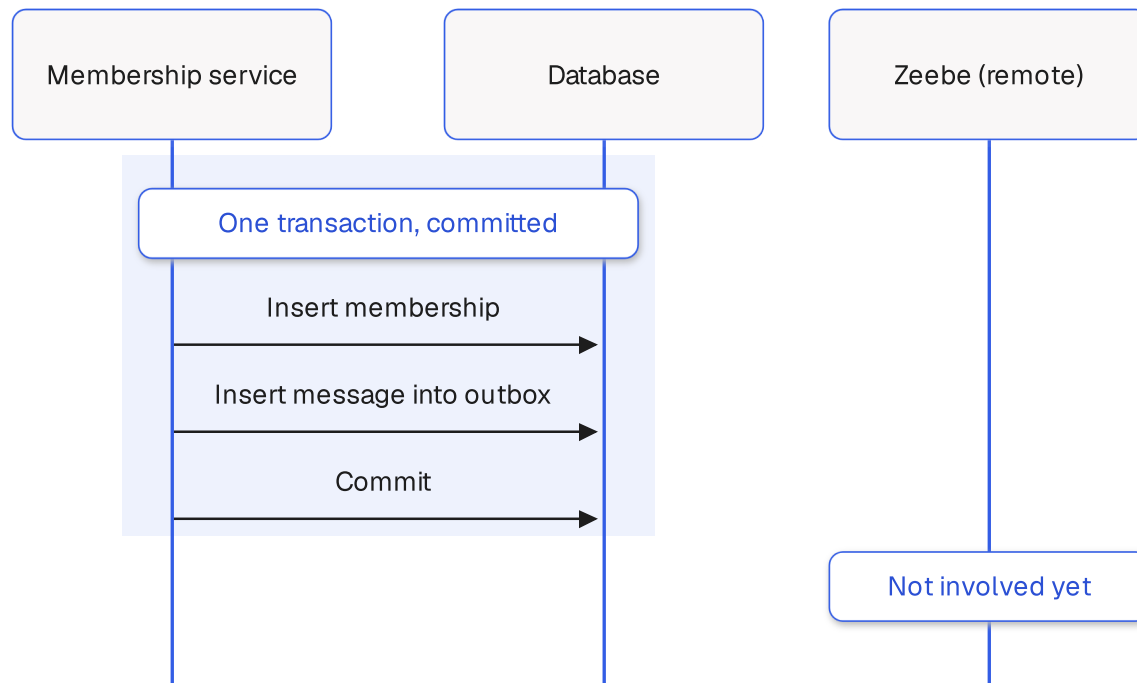
Store the business object and the message together.
Either both or none



Phase 2: outbox processor

Poll and lock open messages, send them to Zeebe,
mark them as processed

Phase 1: data and message, all or nothing



The database says

Membership and message stored

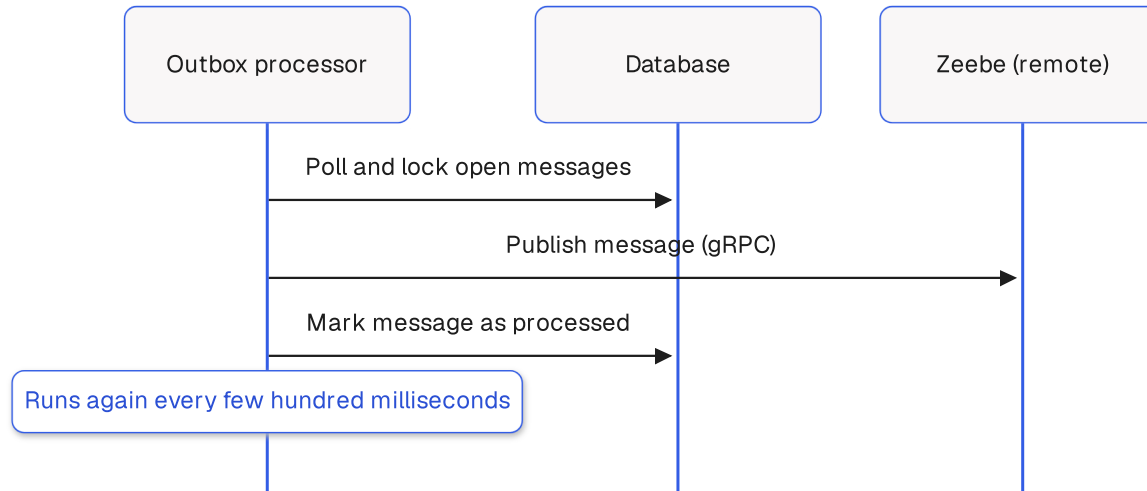
The engine says

Nothing yet

Almost Camunda 7

Instead of starting the process we store the intent to start it

Phase 2: a processor delivers until it worked



The database says

Message processed

The engine says

Process instance running

Delivery guaranteed

A failed call stays in the outbox and is retried

The price: three trade-offs



More infrastructure



Delivery decisions



At least once

- **Sent, but not marked as sent:** the next run sends the message again
- **So not just one goes out:** Zeebe may receive a message twice or more
- **Our system must be able to handle duplicates**

An idea: one lane per key, sent right away



One lane per key



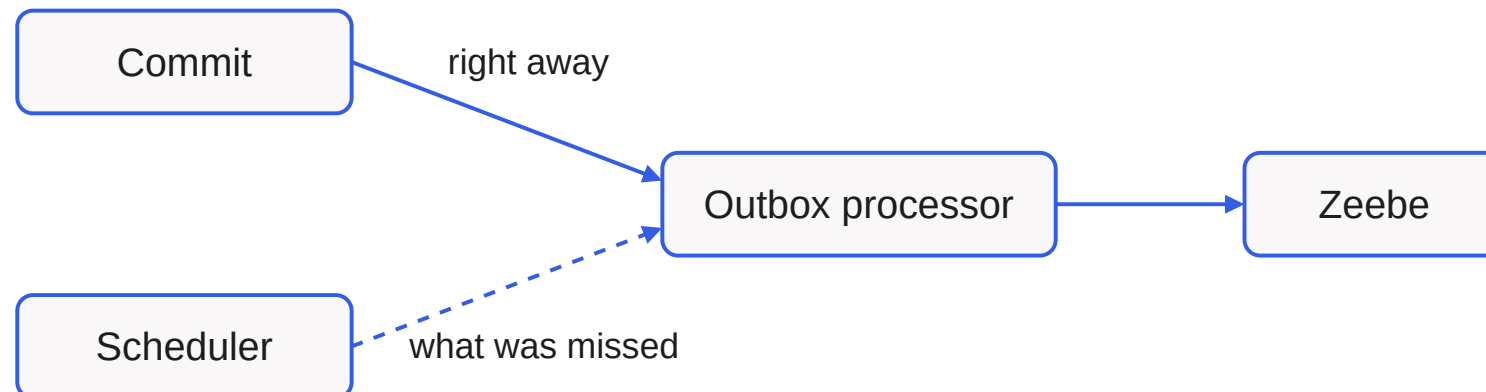
One stuck key



Fast path, safety net

Sent right after the commit

A scheduler catches the rest. Still at least once



Verdict: the way to go, with duplicates

The only attempt that never loses a message



Timing

Solved. Delivery is separated from the business transaction



Delivery

Guaranteed, at least once



Price

A table, a processor, an ordering strategy, and idempotent receivers

THREE ATTEMPTS, ONE WINNER

The outbox wins, but **the gap stays**

— No attempt closes it. Each one trades one risk for another

CHAPTER 5

Mastering idempotency

The price of the winner, paid in three places

Repeating must not hurt

An operation is **idempotent** if one of these holds



Either

Repeating it always produces the same result



Or

Repeating it causes no harm

Three places to build it in

In your code

For workers: a job that comes twice

With Zeebe

For messages: a copy that arrives twice

In the model

For both: fewer places where it hurts

Two ways to handle a repeated job



Run it again



Detect and ignore it

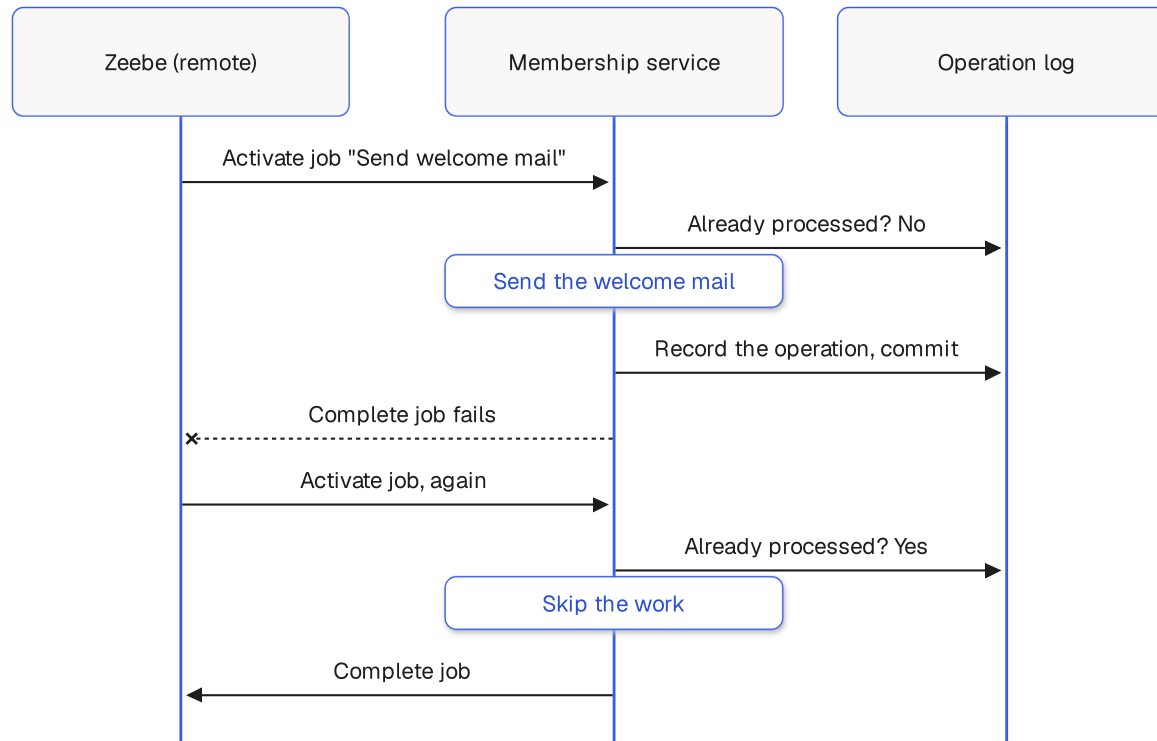
Safe by design

We add a flag to our data model and check it before we act

MEMBERSHIP

id	42
status	CONFIRMED
welcome_mail_sent	true

An operation log: the outbox turned around



Check, execute, record

All three in one transaction

The outbox turned around

We log what came in, not what leaves

Better, not perfect

A third system can still see it twice

Are messages idempotent already?

You might think so. Send a message twice, and the process moves on only once



Always accepted

A valid message is accepted, whether or not an instance waits for it



Buffered for its TTL

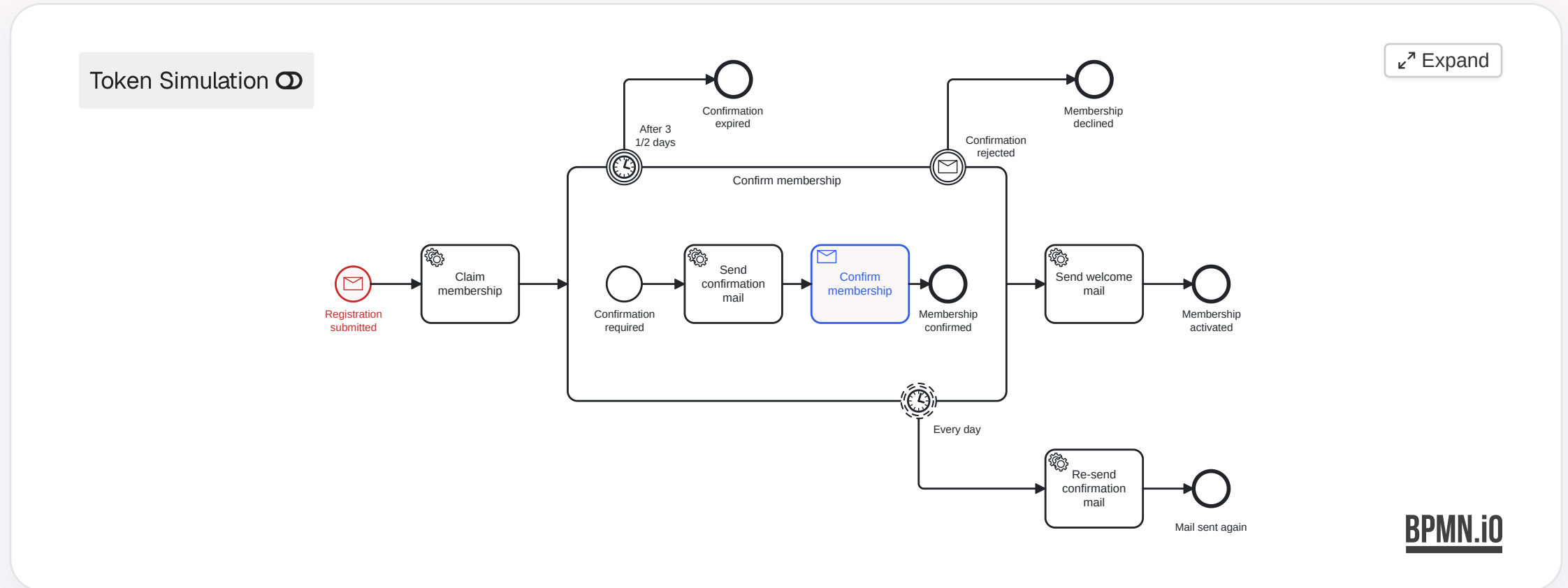
With a time-to-live above zero it waits in the broker until someone subscribes



Correlated once per process

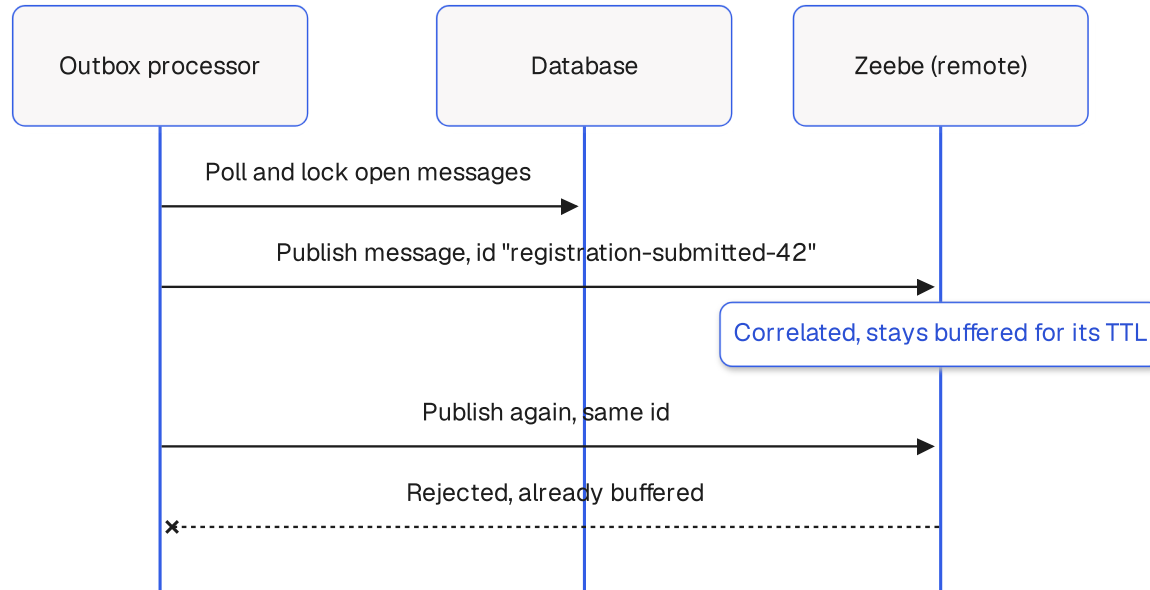
One message is correlated to a process only once, across all its versions

Harmless at the receive task, not at the start



Receive task: copies find nobody waiting. Start event: no key, an instance per copy. Key: one active instance

That is why Zeebe added the messageId



An idempotency key

Sent along with every message

Every later copy

Rejected: same name, key and id

The catch

Only while the first is buffered

One line sets it

MembershipProcessAdapter.kt

KOTLIN

```
override fun submitRegistration(id: MembershipId) {  
    camundaClient.newPublishMessageCommand()  
        .messageName("miravelo.registrationSubmitted")  
        .correlationKey(id.value.toString())  
        .messageId("registration-submitted-`${id.value}`")  
        .timeToLive(Duration.ofHours(1))  
        .variables(variables)  
        .send()  
        .join()  
}
```

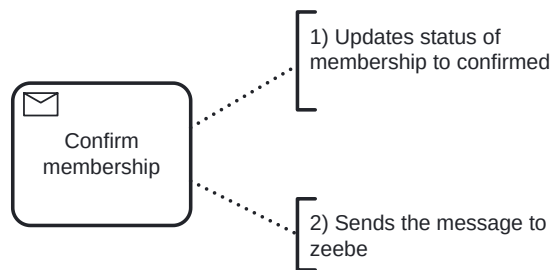
With the Java client, in Spring too. We derive the id from the business key, so every retry sends the same one

Split tasks that do two things

The rule: one task writes to one system. Reading from others is fine

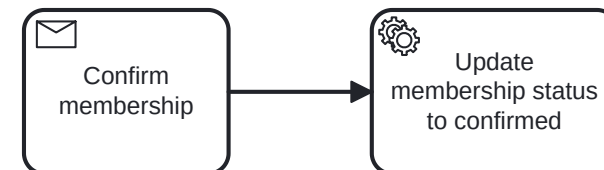
Before

One task: status **and** message

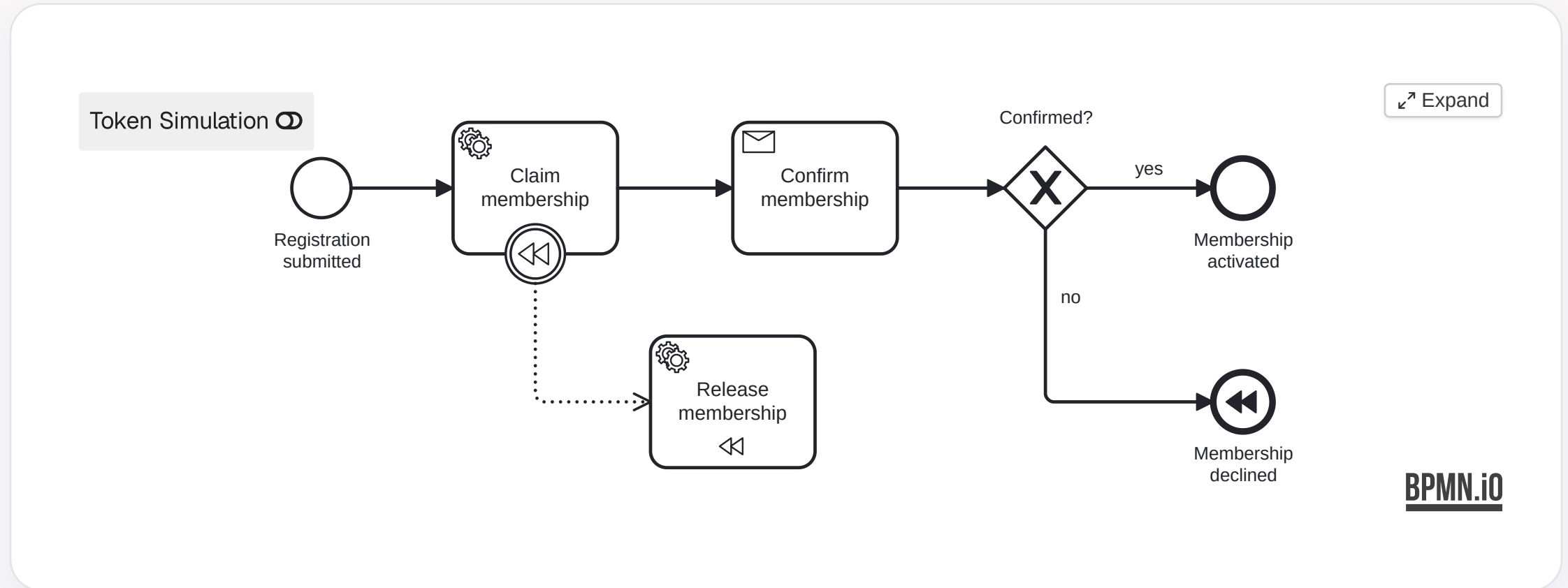


After

One task each: message, then status



An idea: could an undo step help?



Across systems there is no rollback. But BPMN can model the undo, as a compensation

IN YOUR CODE, WITH ZEEBE, IN THE MODEL

None of the three **closes** **the gap**

- Together they make it small enough to handle. And every sender needs an idempotent receiver, not just the outbox

CHAPTER 6

Our Learnings

What to take home, and what to do with it

06

The world has changed

From an ACID world of monoliths into a BASE world of distributed systems

Camunda 7

One database

One transaction

Result: all or nothing, guaranteed

Zeebe

Many components, each with its own state

No shared transaction

Result: distributed transaction problems

It is not a Zeebe problem

Whenever one operation writes to several systems, you have it. The pattern is always the same



Database + Zeebe

What we looked at today



Database + Kafka

The event is out, the commit fails



Database + REST API

The call went through, the commit fails

The good news: the same patterns apply everywhere. And so do their trade-offs

The whole team has to know it

Not everyone has to implement the patterns. But everyone should know what they mean for their area



Developers

Implement the patterns



Product and UX

Design for "not yet consistent"



QA

Test the failure scenarios



Operations

Monitor differently

My tip: a brown bag session. One hour, the problem, the solutions, and what it means for your system

Design your systems for it



1: Detect



2: Prioritize



3: Solve

- **It pays off:** production bugs cost more than the patterns
- **Use the toolbox:** after transaction, outbox, idempotency, by scenario and priority
- **Use synergies:** one outbox for every interaction with Zeebe
- **Test and monitor:** prove that it behaves, and get told when it does not

What would have saved us the tickets



Ticket #4270



Ticket #4271

- **Registered, but no mail at all**
- **Cause:** Zeebe started the process, our commit failed
- **Would have helped:** after transaction or the outbox, so the engine only hears of committed data



THE ONE SENTENCE TO TAKE HOME

**We cannot prevent the
problem. We can design
for it**

 No pattern closes the gap. Each one narrows it, at a price you choose

The End



Code-Examples & other sources

github.com/emaarco/distributed-horcruxes



Information from camunda on the topic

- [Blog: Navigating technical transactions with Camunda 8 and Spring](#)
- [Docs: Message uniqueness](#)
- [Video: youtube.com/watch?v=WRR26jJNh68](https://youtube.com/watch?v=WRR26jJNh68)

CAMUNDA CHAPTER MUNICH

Thank you!

MIRAGON

marco.schaeck@miragon.io